



PYTHON SETS – DETAILED EXPLANATION WITH CONTENTS

◆ Introduction to Sets in Python

- A set in Python is a built-in data type used to store a collection of unique elements.
- Sets are inspired by mathematical sets and are mainly used when we want to:
- Remove duplicate values
- Perform mathematical operations like union, intersection, etc.
- Check membership efficiently

◆ Key Characteristics of Sets

- Sets can store both homogeneous and heterogeneous data types.
- A set is mutable, meaning elements can be added or removed.
- Elements inside a set must be immutable (int, float, string, tuple).
- Sets are unordered, so elements do not follow a fixed position.
- Indexing and slicing are not possible.
- Duplicate values are automatically removed.
- Sets are represented using curly braces { }.
- Nested sets are not allowed, because sets themselves are mutable.

◆ Why Set Elements Must Be Immutable?

Python uses hashing to store elements in a set.

Mutable objects like lists can change their value, which breaks hashing.

Hence:

✗ List inside set → Not allowed

✓ Tuple inside set → Allowed

```
a = {1, 2, (3, 4)} # Valid
```

```
a = {1, 2, [3, 4]} # TypeError
```

◆ Defining a Set

1 Using Curly Braces

```
s = {1, 2, 3, 4}
```

2 Using set() Constructor

```
a = set() # Empty set
```

```
b = set([1, 2, 3, 4]) # From iterable
```

⚠️ {} creates a dictionary, not a set.

◆ Unordered Nature of Sets

Sets do not maintain insertion order.

Elements may appear in a different order every time you print them.

Because of this:

✗ Indexing is not possible

✗ Slicing is not possible

```
a = {10, 20, 30}
```

```
# a[0] → Error
```

◆ Set Contents (What Can a Set Store?)

Data Type Allowance

Data Type	Allowed
int	✓
float	✓
string	✓
boolean	✓
tuple	✓
list	✗
set	✗
dictionary	✗

◆ Set Methods – Adding Elements

1 add()

Adds only one element at a time.

```
a = {1, 2, 3}
```

```
a.add(4)
```

```
a.add("hello")
```

2 update()

Adds multiple elements

Accepts only iterable objects

```
a = {1, 2, 3}
```

```
a.update([4, 5, 6])
```

```
a.update("hi")
```

✗ Error example:

```
a.update(5) # int is not iterable
```

◆ Set Methods – Removing Elements

3 pop()

Removes a random element

```
a = {1, 2, 3}
```

```
a.pop()
```

4 remove()

Removes a specified element

Raises KeyError if element is not present

```
a.remove(2)
```

5 discard()

Same as remove()

✗ No error if element not present

```
a.discard(10)
```

6 clear()

Removes all elements

```
a.clear()
```

7 copy()

Creates a duplicate set with a different memory location

```
b = a.copy()
```

◆ Set Operations (Mathematical Concepts)

8 Union

Combines elements from both sets (no duplicates)

$a \mid b$

`a.union(b)`

9 Intersection

Common elements between sets

$a \& b$

`a.intersection(b)`

10 Difference

Elements present in first set but not in second

$a - b$

`a.difference(b)`

1 1 Symmetric Difference

Uncommon elements from both sets

$a \wedge b$

`a.symmetric_difference(b)`

- ◆ Update Variants (Memory-Based)
- ◆ Advantages of Using Sets

- ✓ Removes duplicates automatically
- ✓ Faster membership testing
- ✓ Supports powerful mathematical operations
- ✓ Useful in data cleaning and analysis

◆ Set Update Methods (Modify the Original Set)

These methods **do not create a new set**.

They **change the existing (original) set in the same memory location**.

1 intersection_update()

👉 What it does:

- Keeps **only the common elements**
- Removes all other elements from the original set

Syntax:

```
set1.intersection_update(set2)
```

Example:

```
a = {1, 2, 3, 4}
```

```
b = {3, 4, 5, 6}
```

```
a.intersection_update(b)
```

```
print(a)
```

Output:

```
{3, 4}
```

Explanation:

- Common elements between a and b $\rightarrow$ {3, 4}
 - a is updated with only these values
-

2 difference_update()

What it does:

- Removes elements from the original set that are **present in another set**
- Keeps **only unique elements of the first set**

Syntax:

```
set1.difference_update(set2)
```

Example:

```
a = {1, 2, 3, 4}
```

```
b = {3, 4, 5}
```

```
a.difference_update(b)
```

```
print(a)
```

Output:

```
{1, 2}
```

Explanation:

- Elements common in b $\rightarrow$ {3, 4}
 - These are removed from a
 - Remaining elements $\rightarrow$ {1, 2}
-

3 symmetric_difference_update()

👉 What it does:

- Keeps **only uncommon elements from both sets**
- Removes all common elements
- Updates the original set

📌 Syntax:

```
set1.symmetric_difference_update(set2)
```

✅ Example:

```
a = {1, 2, 3}
```

```
b = {3, 4, 5}
```

```
a.symmetric_difference_update(b)
```

```
print(a)
```

● Output:

```
{1, 2, 4, 5}
```

🧠 Explanation:

- Common element → {3} (removed)
- Uncommon elements → {1, 2, 4, 5}
- a is updated with these values

◆ Difference Between Normal vs Update Methods

Method	Creates New Set	Modifies Original
intersection()	✅	❌
intersection_update()	❌	✅
difference()	✅	❌
difference_update()	❌	✅
symmetric_difference()	✅	❌
symmetric_difference_update()	❌	✅

◆ Set Relationship Methods

These methods **do not change the set**.
They only return **True or False**.

issubset()

What it does:

Checks whether **all elements of one set are present in another set**

Syntax:

```
set1.issubset(set2)
```

Example:

```
a = {1, 2, 3, 4, 5}
```

```
b = {2, 3}
```

```
print(b.issubset(a))
```

Output:

True

Explanation:

- Every element of b exists in a
 - So b is a subset of a
-

5 **issuperset()**

What it does:

Checks whether a set **contains all elements of another set**

Syntax:

```
set1.issuperset(set2)
```

Example:

```
a = {1, 2, 3, 4, 5}
```

```
b = {2, 3}
```

```
print(a.issuperset(b))
```

Output:

True

Explanation:

- a contains all elements of b
 - So a is a superset of b
-

isdisjoint()

What it does:

Checks whether **two sets have NO common elements**

Syntax:

```
set1.isdisjoint(set2)
```

Example 1 (No common elements):

```
a = {1, 2}
```

```
b = {3, 4}
```

```
print(a.isdisjoint(b))
```

Output:

```
True
```

Example 2 (Common element exists):

```
a = {1, 2}
```

```
b = {2, 3}
```

```
print(a.isdisjoint(b))
```

Output:

```
False
```

Explanation:

- 2 is common → sets are NOT disjoint
-

Quick Summary

- ✓ `intersection_update()` → keeps common elements
- ✓ `difference_update()` → removes common elements
- ✓ `symmetric_difference_update()` → keeps uncommon elements
- ✓ `issubset()` → checks part-of relationship
- ✓ `issuperset()` → checks contains relationship
- ✓ `isdisjoint()` → checks no common elements

◆ Final Summary

- Set is an unordered collection of unique elements
- Elements must be immutable
- No indexing or slicing
- Ideal for mathematical and logical operations
- Frequently used in Python data processing